

Lösungsblatt: Einführung in verteilte Systeme

Modul 321 – Lektion 1

Hinweis: Dieses Lösungsblatt enthält Musterlösungen und mögliche Erwartungshorizonte. Je nach Begründung, Beispielwahl und fachlicher Sauberkeit sind auch andere Antworten sinnvoll.

1. Grundbegriffe erklären

Begriff	Mögliche Lösung
Verteiltes System	Ein verteiltes System besteht aus mehreren unabhängigen Rechnern oder Diensten, die über ein Netzwerk zusammenarbeiten. Für Benutzerinnen und Benutzer wirkt es oft wie ein einziges System, obwohl intern mehrere Komponenten beteiligt sind.
Cloud-Native	Cloud-Native beschreibt eine Art, Software so zu entwickeln und zu betreiben, dass sie gut in Cloud-Umgebungen funktioniert. Typisch sind Container, automatische Skalierung, verteilte Dienste und automatisierte Deployments.
Cloud Functions	Cloud Functions sind kleine Programme, die als Reaktion auf ein Ereignis ausgeführt werden, zum Beispiel nach einem Upload oder einer Anfrage. Sie laufen meist nur dann, wenn sie gebraucht werden.
XaaS	XaaS bedeutet “Anything as a Service” und beschreibt, dass IT-Leistungen als Dienst über das Netzwerk bereitgestellt werden. Dazu gehören zum Beispiel Software, Plattformen, Speicher oder Rechenleistung.

2. Beispiele zuordnen

1. Verteiltes System
2. Cloud Functions
3. SaaS / XaaS
4. Cloud-Native
5. Verteiltes System oder XaaS, je nach Begründung; als *passendster* Begriff hier meist Verteiltes System
6. Cloud Functions

Mögliche Begründungen:

- Beispiel 1 ist ein verteiltes System, weil mehrere getrennte Komponenten zusammenarbeiten müssen.
- Beispiel 3 ist SaaS/XaaS, weil die Firma eine fertige Dienstleistung nutzt, ohne die Software selbst zu betreiben.
- Beispiel 4 ist Cloud-Native, weil die Anwendung gezielt für den Betrieb in einer Cloud-Umgebung entworfen wurde.

3. XaaS konkretisieren

Abkürzung	Bedeutung	Beispiel
IaaS	Infrastructure as a Service	Virtuelle Server bei AWS EC2, Azure Virtual Machines oder Hetzner Cloud
PaaS	Platform as a Service	Heroku, Google App Engine oder Render
SaaS	Software as a Service	Microsoft 365, Google Docs, Dropbox
FaaS	Function as a Service	AWS Lambda, Azure Functions, Google Cloud Functions

4. Kurzvergleich

1. Cloud-Native beschreibt, wie Software gebaut und betrieben wird; XaaS beschreibt, wie Leistungen als Dienst angeboten werden.
2. Ein verteiltes System ist komplexer, weil mehrere Komponenten koordiniert werden müssen und über das Netzwerk kommunizieren.
3. Cloud Functions haben den Vorteil, dass Code automatisch bei Bedarf ausgeführt wird und nicht dauerhaft laufen muss.
4. Netzwerkkommunikation ist fehleranfälliger, weil Verzögerungen, Ausfälle oder Verbindungsprobleme auftreten können.
5. Das Netzwerk verbindet die einzelnen Teile des Systems und ermöglicht Datenaustausch und Zusammenarbeit.

5. Alltagstransfer

Beispiel: Spotify

1. Spotify kann als verteiltes System betrachtet werden, weil App, API, Benutzerverwaltung, Musikdatenbank und Streaming-Infrastruktur nicht als einzelnes Programm laufen.
2. Getrennte Dienste könnten Suche, Empfehlungen, Benutzerkonten, Musikstreaming und Abrechnung sein.
3. XaaS könnte zum Beispiel bei Datenbanken, Speicher, Monitoring oder Zahlungsdiensten genutzt werden.
4. Cloud Functions könnten beim Erstellen von Vorschaubildern, beim Versenden von Benachrichtigungen oder beim Verarbeiten neuer Uploads eingesetzt werden.

5. Vorteile sind bessere Skalierbarkeit, klare Aufgabentrennung und einfachere Erweiterbarkeit.
6. Risiken sind grössere Komplexität, Netzwerkabhängigkeit, schwierigere Fehlersuche und mögliche Ausfälle einzelner Dienste.

Hinweis: Andere alltagsnahe Anwendungen sind natürlich ebenfalls korrekt, wenn die Begründung schlüssig ist.

6. Mini-Analyse eines Szenarios

1. Mögliche Komponenten: Webanwendung, Datenbank, Datei- oder Objektspeicher, Virenskan-Dienst, Nachrichtenfunktion, Videokonferenz-Dienst, Authentifizierung.
2. Auf ein verteiltes System weisen insbesondere externer Cloud-Speicher, eingebundener Videodienst und der automatische Virenskan hin, weil mehrere getrennte Dienste zusammenarbeiten.
3. XaaS wird beim externen Cloud-Speicher und beim Videokonferenz-Dienst genutzt, da diese Leistungen als fertige Dienste bezogen werden.
4. Eine Cloud Function oder ein ähnlicher Mechanismus könnte nach jedem Datei-Upload automatisch den Virenskan starten.
5. Vorteile: bessere Skalierbarkeit, Nutzung spezialisierter Dienste. Herausforderungen: Abhängigkeit von Drittanbietern, komplexere Integration, Datenschutz- und Netzwerkfragen.

7. Skizze einer einfachen Architektur

Mögliche Komponenten einer Online-Terminbuchung:

- Frontend im Browser oder als App
- Backend / API
- Datenbank für Termine, Benutzer und Verfügbarkeiten
- Externer Mail- oder SMS-Dienst für Bestätigungen
- Optional eine automatisch ausgelöste Funktion für Erinnerungen

Mögliche Antworten:

1. Frontend, Backend, Datenbank und externer Benachrichtigungsdienst laufen wahrscheinlich auf verschiedenen Systemen.
2. Die Kommunikation zwischen Browser und API, zwischen API und Datenbank sowie zwischen API und externem Mail-/SMS-Dienst läuft über das Netzwerk.
3. Am ehesten müsste das Backend oder die API skaliert werden, weil dort viele gleichzeitige Anfragen eintreffen können.

8. Offene Recherche-Aufgabe für schnelle Studierende

Beurteilungskriterien statt einer einzigen Musterlösung:

- Das gewählte Beispiel ist konkret und thematisch passend.
- Die wichtigsten Komponenten oder Dienste werden nachvollziehbar beschrieben.
- Es wird klar erklärt, warum das Beispiel zu verteilten Systemen, Cloud-Native oder XaaS passt.
- Vorteile und Herausforderungen werden nicht nur aufgezählt, sondern knapp erläutert.
- Bei einer mündlichen Vorstellung sind Fachbegriffe korrekt und verständlich eingesetzt.

Geeignete Themenbeispiele:

- Netflix und Content Delivery Networks
- WhatsApp oder Signal und Nachrichtenübertragung auf viele Server
- Microsoft Teams oder Zoom als Kombination mehrerer Cloud-Dienste
- AWS Lambda oder Azure Functions als Beispiel für FaaS

Didaktischer Hinweis

Das Lösungsblatt ist bewusst knapp gehalten. In der Besprechung kann je nach Leistungsstand der Klasse vertieft werden, etwa bei Themen wie Skalierung, Fehlertoleranz, Microservices oder Event-basierter Verarbeitung.